Simuler pour animer

Eric Maisel

Printemps 2025

1 Objectif

Pour assurer leur crédibilité il faut que les mondes virtuels soient "peuplés". S'il faut que les objets de ces mondes soient incarnés (on doit pouvoir les voir, éventuellement les entendre et pourquoi pas les toucher) il faut également qu'ils puissent évoluer dans le temps, se déplacer, être animé.

On peut douter que cela soit suffisant. En faisant cela on n'a fait que rajouter une dimension. On passe juste d'une scène statique en 3d à une scène statique en 4d. En fait la crédibilité d'un monde virtuel passe par le fait que les objets qui s'y trouvent réagissent à notre présence (aspect réactif) voir qu'ils adaptent leur comportement à notre présence pour continuer à chercher à atteindre un objectif qui leur est propre (aspect pro-actif).

Une approche pour arriver à ces comportements consiste à contrôler les objets au moyen de forces définies par l'interaction entre les objectifs des objets du monde virtuel et leur environnement - environnement contenant une représentation de l'utilisateur immergé dans ce monde virtuel.

Ce sont les bases de cette approche qui font l'objet de cette deuxième partie du cours.

2 Architecture de contrôle flexible

2.1 Objectif

Il s'agit de proposer une architecture logicielle qui permette de faire évoluer facilement un monde virtuel, tant à la conception que lors de simulations.

2.2 Principe

De base les objets d'un monde virtuel sont décrits par un graphe de scène. Les objets, représentés par des **meshes** sont placés les uns par rapport aux autres au moyen de transformations affines (compositions de translations et de rotations).

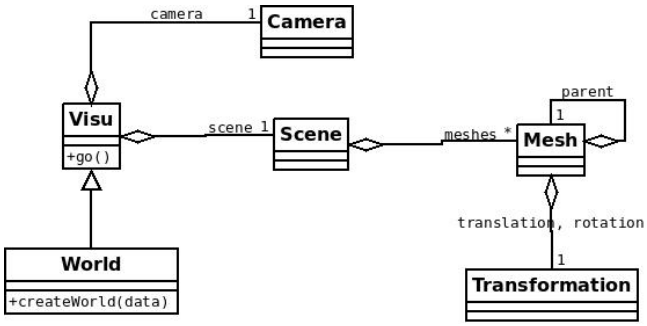

Figure 1: Graphe de scène

Afin de pouvoir contrôler les objets 3d on ajoute des "gestionnaires" d'objets 3d. Un gestionnaire représente un objet du monde virtuel et son état. Il est caractérisé par :

- un identifiant
- une incarnation dans le monde virtuel, représentée par un objet 3d
- une position et une orientation dans le monde virtuel
- une méthode de mise à jour (`update(dt)`)

Ces objets sont implémentés par la classe `Entity`.

Un des intérêts de cette approche réside dans la possibilité de placer la logique de simulation non pas dans la classe `Visu` ou une de ses sous-classes mais au niveau des objets de la simulation eux-même, c'est à dire dans les méthodes `update` de la classe `Entity` ou de ses sous-classes.

Dans cette architecture la simulation est gérée par les classes (ou sous-classes):

- `Visu` : assure le rendu des objets 3d
- `Simu` : gère les objets du monde virtuel et invoque les codes de changement de l'état de ces objets (méthode `update`)

- `World` : crée les objets du monde virtuel

Si on se contente de cette première définition des entités la seule possibilité de créer des entités ayant des aspects et surtout des comportements différents consiste à créer une sous-hiérarchie de classe dont la racine est la classe `Entity`.

Prenons comme exemple un jeu d'aventure. Celui-ci fait intervenir des chevaliers dotés d'une épée et des chevalier équipés d'un bouclier. On aura donc deux sous-classes dérivant de la classe `Entity` : les classes `KnightWithSword` et `KnightWithShield`. Pour créer des chevaliers ayant à la fois une épée et un bouclier il faudra créer une nouvelle classe `KnightWithSwordAndShield`. Cette nouvelle classe va soit hériter de `KnightWithSword` et `KnightWithShield` (avec tous les problèmes liés à l'héritage multiple) soit hériter de `Entity` (avec une duplication de code des deux classes citées plus haut).

Dans les deux cas cela va poser des problèmes :

- A la conception : une explosion combinatoire des classes de façon à pouvoir tenir compte de tous les cas possibles
- A l'exécution : impossibilité d'ajouter dynamiquement de nouveaux comportements à des entités.

Aussi on va améliorer cette architecture en ajoutant de nouveaux types de classes : les instances de la classe `Component`, appelées naturellement des composants.

On peut considérer dans un premier temps ces composants comme des contrôleurs associés à des objets. A chaque pas de temps ils sont exécutés et peuvent modifier des attributs (les leurs ou des attributs de l'entité qu'ils contrôlent). La méthode `update` de l'entité à laquelle ces composants sont associés est appelée régulièrement et utilise les valeurs de ces attributs afin de se mettre à jour (en particulier le placement de l'objet 3d qui incarne dans le monde virtuel cette entité). L'exemple de code suivant va :

1. créer un matériau de couleur rouge et l'enregistrer sous le nom "rouge"
2. créer une entité de type "entity" enregistrée sous le nom de "sphere01"
3. associer à cette entité un composant permettant de lui associer une incarnation dans le monde virtuel
4. associer à cette entité un composant qui va le placer dans l'espace du monde virtuel en lui appliquant une translation de vecteur de coordonnées $(3, 0.2, 2)$.

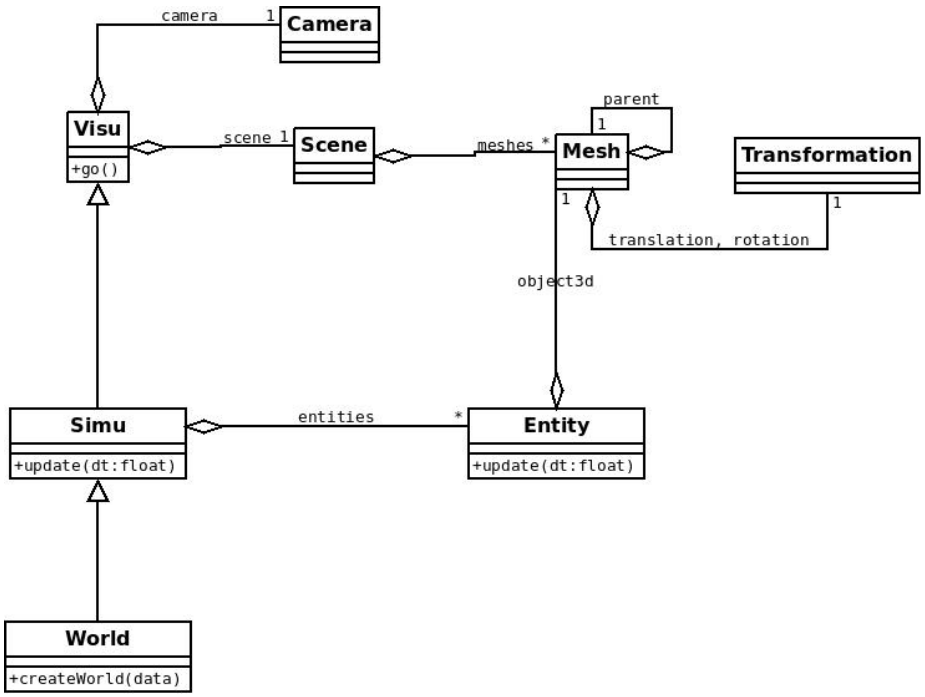

Figure 2: Entités

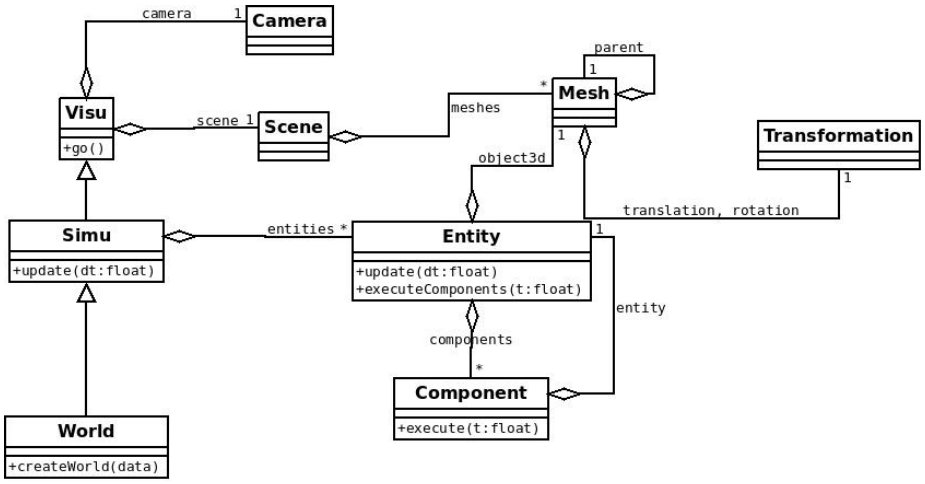

Figure 3: Entités et composants

```
class World {

  constructor(){...}

  createWorld(data){

    registerAsset("rouge",PRIMS.standardMaterial,{couleur:[1,0,0]}) ;
    ...
    const e0 = createEntity("sphere01", ENTITIES.entity, {}) ;
    e0.add(COMPS.sphere,{diameter:1, material:"rouge"}) ;
    e0.add(COMPS.position, {"x":3, "y":0.5, "z":2}) ;
    ...
  }
}
```

Il existe un chargeur de fichiers décrivant la scène (défini dns) qui permet de charger le fichier JSON suivant pour avoir le même résultat :

```
{
  assets : {
             "rouge" : {"foo"  : "standardMaterial,
                        "data" : {"couleur":[1,0,0]}
                       }
           },
  entities : {
   "sphere01":{
                           "type":"entity",
                           "data": {},
                           "components":[
                                 {
                                   "type":"sphere",
                                   "data":{"diameter":0.2,"material":"rouge}
                                 },
                                 {
                                   "type":"position",
                                   "data":{"x":1,"y":2,"z":-0.2}
                                 }
                               ]
                           },
             }
}
```

2.3 Exercices

Exercice 1

Lisez le fichier `scene.json` et comprenez le format utilisé pour décrire un monde virtuel.

Exercice 2

Comprenez le lien entre le fichier `scene.json` et les fichiers `js/world.js`, `js/entities/entities.js`, `js/components/components.js` et tous les autres fichiers associés.

Exercice 3

Exprimez la scène que vous avez déjà définie dans ce format.

Exercice 4

Ajoutez un guide virtuel. Ce guide est incarné par un objet de façon à ce qu'on puisse distinguer sa position et son orientation. Ce guide doit posséder son propre matériau.
Pour obtenir ce résultat :

1. ajoutez une fonction `creerGuide(nom, params, scene)` dans le fichier `js/prims.js`. Une référence sera affectée à `PRIMS.guide`.
2. dans le fichier `components/guide.js` définissez la classe `Guide` (sous-classe de `Component`) qui permet de créer l'incarnation du guide.
3. Créez une entité dont l'incarnation est définie par le composant `Guide`. Placez ce guide dans votre scène.

On veut pouvoir créer un guide ainsi :

```
{
  ...
  "entities" : {
                "tux" : {
                          "type" : "entity",
                          "data" : {},
                          "components" : [
                             {"type" : "guide", "data":{ ...}},
                             {"position" : {"x":3,"z":5}}
                          ]
                        },
                ...
              }
}
}
```

3 Contrôle cinématique

3.1 Objectif

Contrôler le mouvement d'un objet à travers sa vitesse.

3.2 Principe

L'entité `Kine` dérive directement de la classe `Entity`. Le déplacement de l'objet est contrôlé par la vitesse (attribut `velocity`) ou par son accélération (attribut `acceleration`).

L'évolution dans le temps de la position de l'entité et de sa vitesse sont définis par les expressions suivantes :

$$\begin{cases} p(t+dt) & = & p(t) + \vec{v}(t)dt \\ \vec{v}(t+dt) & = & \vec{v}(t) + \vec{\gamma}(t)dt \end{cases}$$

Ce qui conduit à l'implémentation suivante

```
class Kine extends Entity {

  constructor(name, data, sim){
    this.name = name ;
    this.sim  = sim ;
    this.position     = BABYLON.Vector3.Zero() ;
    this.velocity     = BABYLON.Vector3.Zero() ;
    this.acceleration = BABYLON.Vector3.Zero() ;
  }

  update(dt){
    this.velocity.scaleAndAddToRef(dt, this.position) ;
    this.acceleration.scaleAndAddToRef(dt, this.velocity) ;

    if(this.object3d){
      this.object3d.position.copyFrom(this.position) ;
    }
  }
}
```

La vitesse d'un objet implémenté par la classe `Kine` est définie par un composant. En voici un exemple :

```
class BrownianMotion extends Component {

  constructor(data, entity) {
    super(data, entity) ;
    this.register(0.1) ;
    this.p0   = data.p0   || 0.1 ;
    this.vMax = data.vMax || 1.0 ;
  }
```

```
  execute(t){
    const p = Math.random() ;
    if(this.p0>p){
      const vx = (0.5-Math.random())*this.vMax ;
      const vz = (0.5-Math.random())*this.vMax ;
      this.entity.velocity.copyFromFloats(vx,0.0,vz) ;
    }
  }
}
```

Si la classe `BrownianMotion` est associé dans le catalogue `COMPS` (fichier `js/components/components.js`) au nom `brownianMotion` alors une sphère rouge se déplaçant de façon aléatoire peut être alors décrite, dans le fichier `scene.json` de la façon suivante :

```
{
  assets : {
            "rouge" : {
                        "foo"  : "standardMaterial,
                        "data" : {"couleur":[1,0,0]}
                       }
          },
  entities : {
              "sphere01":{
                          "type":"entity",
                          "data": {},
                          "components":[
                                 {
                                   "type":"sphere",
                                   "data":{"diameter":0.2,"material":"rouge"}
                                 },
                                 {
                                   "type":"position",
                                   "data":{"x":1,"y":2,"z":-0.2}
                                 },
                                 {
                                   "type":"brownianMotion",
                                   "data":{"vMax":2, "p0":0.2}
                                 }
                                      ]
                         },
            }
}
```

3.3 Exercices

Exercice 1

Etudiez le composant défini dans le fichier `js/components/brownianMotion.js`. Utilisez ce composant pour animer une sphère rouge dans votre scène.

Exercice 2

Définissez un composant `Rebond` dans un fichier `js/components/rebond.js` dont le rôle est de donner une vitesse initiale à un objet et de le garder confiné dans une boite de diagonale d'extrémité $(x_{min}, y_{min}, z_{min}), (x_{max}, y_{max}, z_{max})$.

Ce confinement signifie que la position (x, y, z) de l'objet contrôlé par ce composant satisfait les contraintes suivantes :

$$\begin{cases} x_{min} < x < x_{max} & (1) \\ y_{min} < y < y_{max} & (2) \\ z_{min} < z < z_{max} & (3) \end{cases}$$

Pour satisfaire ces contraintes la vitesse de l'objet (le vecteur de coordonnées (v_x, v_y, v_z)) sera modifié par les modifications suivantes :

- si la contrainte (1) n'est plus satisfaite : v_x devient $-v_x$
- si la contrainte (2) n'est plus satisfaite : v_y devient $-v_y$
- si la contrainte (3) n'est plus satisfaite : v_z devient $-v_z$

Testez ce comportement sur une sphère de façon à ce que :

- cette sphère soit confinée dans une région cubique d'arêtes de longueur 5m placé dans le hall
- la vitesse de cette sphère est aléatoire, de norme comprise entre 1 et 4 ms^{-1}.

```
{
  ...
  "assets"   : { ... }
  "entities" : { ...,
                 "sph01" : {
                             "type" : "entity",
                             "data" : {},
                             "components" : [
                                {"type":"sphere","data":{}},
                                {"type":"position",
                                  "data":{"x":3,"y":1,"z":5}},
                                {"type":"vitesse",
```

```
                                    "data":{"x":2,"y":1,"z":1}},
                                  {"type":"rebond",
                                    "data":{
                                       "xmin":-2.5, "xmax":2.5,
                                       "ymin":0   , "ymax":5.0,
                                       "zmin":-2.5, "zmax:2.5
                                      }}


                                ]
                              }
  }
}
```

Exercice 3

Reprenez l'exercice suivant mais en introduisant dans la même région cubique 10 objets rebondissants.

4 Contrôle dynamique

4.1 Objectif

Le contrôle d'une entité par la vitesse et/ou l'accélération peut s'avérer délicat :

- c'est une tâche qui peut prendre du temps pour ajuster correctement les vitesses et accélérations sur l'ensemble des trajectoires, en particulier quand il s'agit de synchroniser plusieurs entités entre elles
- c'est une tâche qui peut conduire à des résultats peu réalistes

Il s'agit d'une approche dite phénoménologique dans laquelle l'animateur décrit les effets à obtenir.

Une autre solution, dite causale travaille sur le lien entre les causes et les effets. Elle consiste à décrire les conditions initiales du monde virtuel à simuler ainsi qu'a donner les lois, même simplifiées qui décrivent dans ce monde l'évolution de l'état des entités en fonction de leur environnement.

4.2 Principe

L'évolution de l'état d'une entité est provoqué par l'application d'une force. Ainsi si un objet initialement immobile se met à se déplacer c'est qu'on lui a appliqué une force. De la même manière le changement de trajectoire d'un objet qui se déplace en ligne droite avec ne vitesse constante peut être expliqué par l'application d'une force.

Un changement de trajectoire d'une entité correspond à une variation du vecteur vitesse, donc à une accélération. Le lien entre les forces qui s'exercent sur l'entité et cette accélération sont données par la loi fondamentale de la dynamique proposée par Newton :

$$\sum \vec{F_i}(t) = m\vec{\gamma}(t)$$

La mise à jour de la position d'un objet repose sur le schéma de double intégration d'Euler :

$$\begin{cases} p(t+dt) & = & p(t) + \vec{v}(t)dt \\ \vec{v}(t+dt) & = & \vec{v}(t) + \vec{\gamma}(t)dt \\ \vec{\gamma}(t) & = & \frac{1}{m}\sum \vec{F_i}(t) \end{cases}$$

La mise à jour de la position d'un objet s'effectue ainsi :

1. le bilan des forces qui s'exercent sur l'entité est effectué
2. ces forces sont additionnées les unes aux autre et le résultat est assimilé au produit de la masse de l'entité par son accélération
3. connaissant l'accélération de l'entité il est possible de déterminer sa nouvelle position et sa nouvelle vitesse pour un futur très proche.

La classe `Newton` implémente ce principe :

```
class Newton extends Entity {

  constructor(name, data, sim){
    super(name, data, sim);
    this.mass     = data.mass || 1.0 ;
    this.velocity = new BABYLON.Vector3(0,0,0) ;
    this.force    = new BABYLON.Vector3(0,0,0) ;
  }

  applyForce(f){
    this.force.addInPlace(f) ;
  }

  update(dt){
    this.velocity.scaleAndAddToRef(dt,this.position) ;
    this.force.scaleAndAddToRef(dt/this.mass, this.velocity) ;
    this.force.set(0,0,0) ;

    if(this.object3d){
      this.object3d.position.copyFrom(this.position) ;
    }

  }
}
```

Les forces qu'on applique sont implémentées par des composants. Ainsi une force horizontale dont les variations de direction et d'intensité sont aléatoires va pouvoir être codée de la façon suivante :

```
class Alea extends Component {
  constructor(data, actor){
    this.deltaAngle = data.deltaAngle || 0.1 ;
    this.deltaIntensity = data.deltaIntensity || 0.1 ;
    this.angle = 0.0 ;
    this.intensity = 0.0 ;
    this.force = new BABYLON.Vector3(0,0,0) ;
    this.register() ;
  }
  execute(t){
    this.angle += (0.5-Math.random())*this.deltaAngle ;
    this.intensity += (0.5-Math.random())*deltaIntensity ;
    if(this.intensity < 0)
      this.intensity = 0 ;
    this.force.set(this.intensity*Math.cos(this.angle),
                   0,
                   this.intensity*Math.sin(this.angle));
    this.entity.applyForce(this.force) ;
  }
}
```

4.3 Exercices

Exercice 1

Proposez un composant `Vent` qui permet de simuler le vent qui souffle en rafale.

Testez ce composant sur un guide en faisant en sorte que le guide s'oriente dans le sens de son déplacement.

Testez ce comportement sur deux guides identiques placés l'un à coté de l'autre. Qu'observez vous ?

Exercice 2

Modifiez le composant proposé dans l'exercice précédent de façon à ce que deux objets proches subissent au même instant la même force due au vent.

Exercice 3

Proposez un composant `Frottement` qui simule les frottements de la surface d'un objet dans le milieu dans lequel il évolue.

On basera la mise en oeuvre du composant `Frottement` sur les points suivants :

- La force de frottement s'oppose au déplacement de l'entité contrôlée
- La norme de la force de frottement est proportionnelle à la vitesse de déplacement de l'entité contrôlée (à l'arrêt il n'y a pas de frottement)
- La viscosité du fluide dans lequel se déplace l'entité considérée, l'état de sa surface en contact avec le fluide ainsi que sa section perpendiculaire au déplacement sont représentés par des facteurs multiplicatifs (respectivement ρ, k_f et A)

On peut modéliser cette force de frottement par :

$$\vec{F_f} = -\rho k_f A^2 \vec{v}$$

En simplifiant :

$$\vec{F_f} = -\rho k \vec{v}$$

5 Steering

5.1 Objectif

Le principe de causalité évoqué dans la section précédente est, en pratique, difficile à mettre en oeuvre. Il n'est pas toujours facile de prévoir les effets à des conditions initiales particulières. On cherche donc à revenir à des approches dans lesquelles on part des effets mais non pas pour les appliquer directement en terme de position et:ou de vitesse mais

- pour en déduire les forces à appliquer pour les obtenir
- et ceci en continue pour corriger des erreurs ou s'adapter à des environnements dynamiques.

Le steering proposé par Reynolds est une de ces approches.

5.2 Principe

5.2.1 Autonomie

Le terme "agent autonome" fait référence généralement à des objets qui sont capables de faire leurs propres choix pour agir dans un environnement sans que cette décision soit prise par un leader ou par un plan global.

Dans les mondes virtuels un agent sera dit autonome s'il possède un objectif qui lui est propre et qu'il dispose en lui même des ressources logicielles nécessaires pour réaliser cet objectif malgré les changement non déterminés a priori de cet environnement.

On se limitera ici en terme d'actions aux déplacements dans l'espace mais on devrait pouvoir étendre les résultats obtenus à d'autres types d'actions (modification d'aspects, mouvements fins, actes de langage, ...).

Ces actions vont être réalisées selon le principe vu précédemment : tout mouvement est causé par l'application de forces. Les agents considérés vont devoir, suite aux décisions prises pour atteindre un objectif, s'appliquer des forces (forces motrices). Ceci dans un environnement qui lui également leur applique des forces.

Un agent autonome est une entité logicielle qui suivra donc les règles suivantes :

- un agent autonome a une perception limitée de son environnement
- un agent autonome possède en lui-même un objectif (ou encore but) et des ressources permettant le traitement des informations provenant de son environnement et la prise de décision conduisant à des actions permettant de s'approcher de la réalisation de cette objectif et ceci même dans un environnement dynamique
- Un monde virtuel est un ensemble d'agents autonomes. Il s'agit d'un système distribué : il n'y a ni contrôle centralisé ni plan global

- Dans un tel monde on peut considérer qu'un agent suit un cycle (explicite ou non) "Perception/Décision/Action". L'environnement d'un agent est constitué par les autres agents.

5.2.2 Modélisation

Un monde virtuel est donc un ensemble d'agents en interaction. Chaque agent peut être décrit selon trois niveaux :

Niveau stratégique : ce niveau permet de sélectionner l'objectif à atteindre

Niveau tactique :ce niveau permet de sélectionner les actions à effectuer afin d'atteindre cet objectif. Cela conduit à calculer les forces à appliquer à l'agent afin d'atteindre l'objectif

Niveau locomotion : : il s'agit de reproduire les mouvements fins qui correspondraient dans le monde réel à la production des forces évoquées ci-dessus (par exemple mouvement de membres).

5.2.3 Steering et forces de steering

Le steering est le principe qui va être utilisé pour lier un objectif élémentaire à atteindre et les forces à appliquer afin de pouvoir l'atteindre. Par objectif élémentaire on entend ici : atteindre un point, éviter des obstacles, ... par rapport à d'autres objectifs plus abstraits tels que : "admirer un tableau", "aller chercher une arme", ... qui peuvent se décomposer en plusieurs objectifs élémentaires.

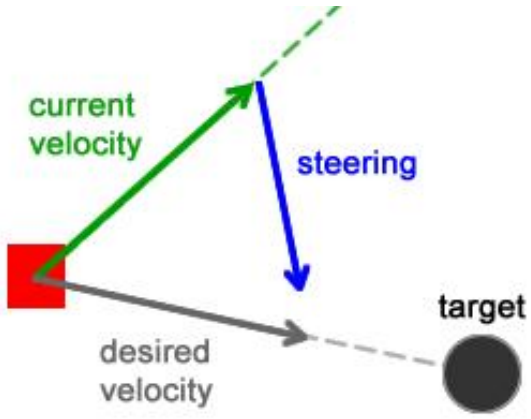

Figure 4: Principe du steering

Considérons un agent doté d'un objectif. Cet agent est situé dans l'espace c'est à dire placé en une position P et orienté selon une direction $\vec{D}$. Il est animé par une vitesse $\vec{v_c}$, sa **vitesse courante**. Dans cette situation atteindre l'objectif considéré suppose qu'il suive une nouvelle vitesse $\vec{v_d}$, la **vitesse désirée**.

Passer d'une vitesse $\vec{v}_c$ à une vitesse $\vec{v}_d$ correspond à subir une accélération (variation de vitesse instantannée) qui peut être assimilée à l'application d'une force dite de steering (cadre de la mécanique Newtonnienne) appelée par la suite $\vec{F}_s$.

Reynolds, l'initiateur du steering, propose dans les travaux qui ont conduits à son article de 1987 "Flocks, Herds and Schools: A Distributed Behavioral Model," dans la revue "Computer Graphics" la relation suivante :

$$\vec{F}_s = \vec{v}_d - \vec{v}_c$$

De façon à rester réaliste on supposera que la norme des vitesses et des forces soient limitées (inférieures à $vMax$ et $fMax$ des valeurs caractéristiques des objets à animer).

Il s'agit donc, étant donnés un agent et un objectif :

1. de calculer la vitesse $\vec{v}_d$ permettant à l'agent de s'approcher de la réalisation de l'objectif
2. d'appliquer la force de steering $\vec{F}_s = k(\vec{v}_d - \vec{v}_c)$ à l'entité à animer

5.2.4 Comportement basique : passer par un point (Seek)

L'exemple de comportement le plus simple est celui qui consiste à faire atteindre/passer un agent par un point C. On appelle ce comportement `Seek`.

L'agent est situé en P , animé par une vitesse $\vec{v}_c$. Il peut atteindre une vitesse maximale de v_M. On cherche à le faire passer par un point cible C.

Le vecteur vitesse désirée $\vec{v}_d$ relie le point P au point C (il faut passer de P à C). On peut alors écrire que :

$$\vec{v}_d = k\vec{PC}$$

Comme l'utilisateur cherche à aller le plus vite possible du point P au point C on peut écrire :

$$\vec{v}_d = \frac{v_M}{|\vec{PC}|}\vec{PC}$$

La force de steering à appliquer sera donc :

$$\vec{F}_s = \frac{v_M}{|\vec{PC}|}\vec{PC} - \vec{v}_c$$

C'est cette force (en tenant compte des limitations des normes de vitesse et de force) qui sera appliquée à l'agent.

En ce qui concerne la mise en oeuvre de ce mécanisme :

- L'agent sera mise en oeuvre par une classe `Newton` dérivant de la classe `Entity`.
- Le comportement permettant de passer par un point cible sera quant à lui mis en oeuvre par une classe `Seek` sous-classe de la classe `Component`.

5.2.5 Arrêt en un point (Arrive)

On constate vite que le principe qui vient d'être vu permet de passer par un point mais pas de s'y arrêter. Généralement l'entité contrôlée dépasse le point visé de façon suffisamment importante pour considérer qu'elle l'a raté : elle se retourne sur elle même et y retourne ... S'ensuit un phénomène d'oscillation autours du point cible.

La solution pour s'arrêter en un point cible consiste à définir autours de ce point une zone de freinage dans laquelle l'entité contrôlée va progressivement diminuer sa vitesse. Ce comportement est appelé `Arrive`.

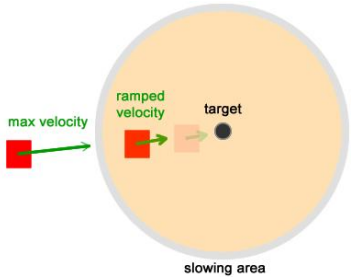

Figure 5: Arrêt en un point

On cherche à faire s'arrêter une entité représentée par un point P et une vitesse $\vec{v}$ en un point C auquel on associe une zone de freinage de rayon d_0.

Le principe est d'utiliser le comportement précédent (comportement `Seek`) mais en faisant varier la valeur de la vitesse maximale v_M en fonction de la distance de l'entité contrôlée à la cible :

$$v_M = \begin{cases} v_0 & si \quad d >= d_0 \\ k\frac{v_0}{d_0}d & si \quad d < d_0 \end{cases}$$

5.2.6 Comportement d'évitement d'obstacles (Avoid)

Le contrôle par steering peut également s'appliquer pour des comportements d'évitement d'obstacle, de suivi de trajectoire, de mur, ... A chaque fois il s'agit, pour un objectif désiré de définir la vitesse à adopter pour atteindre cet objectif et d'en déduire la force de steering à appliquer afin d'atteindre l'objectif.

Dans le cas de l'évitement d'obstacle on va considérer que :

- L'entité contrôlée est représentée par un point P animé d'une vitesse $\vec{v}$
- L'objet à éviter est représenté par un point C et une zône d'évitement de rayon r centrée sur C

Le principe de l'évitement est le suivant : lorsque l'entité contrôlée est trop proche de l'obstacle considéré une force d'évitement lui est appliquée qui l'éloigne de l'obstacle.

Reste à déterminer quand appliquer cette force et quelle force appliquer.

Pour cela on dote l'entité contrôlée d'un palpeur. L'application de la force d'évitement se fait quand le palpeur entre dans la zone d'évitement.

Le palpeur est en fait un point P' placé en avant de l'entité contrôlée dans le sens du déplacement :

$$\vec{PP'} = k\vec{v}$$

La force d'évitement s'applique lorsque

$$||\vec{CP'} < r$$

Un point P'' est calculé : il s'agit d'un point par lequel passé pour éviter l'obstacle. Bien entendu ce point doit être situé hors de la zone d'évitement :

$$P'' = C + (r + d)\frac{\vec{CP'}}{||\vec{CP'}||}, d > 0$$

Un comportement `Seek(`P''`)` est alors appliqué de façon à passer par le point P'' pour éviter l'obstacle.

Bien entendu ce comportement doit être appliqué en continu (mais à quel rythme ?) pour tenir compte des obstacles mobiles.

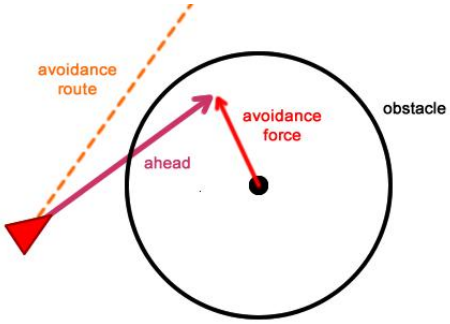

Figure 6: Steering : évitement d'obstacle

5.2.7 Combinaison de comportements

Il est possible d'envisager des comportements complexes résultant de la combinaison de plusieurs comportements élémentaires. Par exemple suivre une trajectoire (comportement C_1), tout en longeant un mur (comportement $C - 2$) et en évitant des obstacles (comportement C_3).

D'après le principe de steering chaque comportement C_i correspond une force de steering $\vec{F_i}$ appliquée à l'entité contrôlée.

La combinaison des comportements se traduit par une combinaison linéaire des forces associées.

Par exemple ici :

$$\vec{F_s} = k_1\vec{F_1} + k_2\vec{F_2} + k_3\vec{F_3}$$

Avec $k_1 + k_2 + k_3 = 1$

5.3 Exercices

Exercice 1

Proposez un composant `Trajectoire` qui permet de passer par $N - 1$ points avant de s'arréter sur le N^{eme}.

Exercice 2

Proposez un composant `Evitement` qui permet d'éviter un obstacle.

Exercice 3

Faîtes en sorte que la caméra puisse être considérée comme une entité. Placez dans votre monde virtuel plusieurs sphères qui seront considérées comme des amers (sphères blanches semi-transparentes). Quand l'utilisateur clique sur un amer il s'y déplace en utilisant les principes du steering.

6 Simulation de foule

6.1 Objectif

On veut simuler un système complexe tel qu'une foule. Le comportement d'un tel système ne peut pas se limiter à la somme de comportements individuels.

Un tel système est caractérisé par les points suivants :

- le comportement de chaque élément dépend de celui des autres
- l'environnement n'est pas déterminé a priori
- les règles qui définissent le comportement des membres du système sont identiques (tout en acceptant une variabilité sur quelques paramètres)

6.2 Principe

Reynolds, dans son article de 87 propose trois règles permettant de faire vivre un groupe d'agents (une foule ?) Ces règles permettent à la fois d'assurer la cohésion du groupe tout en évitant les collisions entre ses différents membres.

Pour rester dans le même esprit que ce qui a été vu précédemment : chaque règle qui régit le comportement d'un élément du système conduit à l'application d'une force à cet élément.

Trois règles sont appliquées :

- une règle de séparation $\vec{F_s}$: elle permet d'éviter les collisions entre les différents éléments du système
- une règle de cohésion $\vec{F_c}$ qui permet aux différents éléments du système de rester grouper
- une règle d'alignement $\vec{F_a}$ qui permet aux différents éléments du système de se déplacer dans la même direction avec la même vitesse.

On applique à chaque objet une force

$$\vec{F} = k_s\vec{F_s} + k_c\vec{F_c} + k_a\vec{F_a}$$

6.2.1 Séparation

Cette règle permet d'éviter les collisions entre éléments du système considéré. Elle consiste à appliquer sur l'élément O_i une force de répulsion engendrée par les éléments O_j du voisinage de O_i.

P_i (resp. P_j) représente O_i (resp. O_j). On peut exprimer cette force de la façon suivante :

$$\vec{F_s} = \sum_{O_j \in voisinage(O_i)} \frac{1}{||\vec{Pj}Pi||^2} \vec{Pj}Pi$$

En première approximation le voisinage de O_i est une sphère de rayon r_e centrée sur P_i.

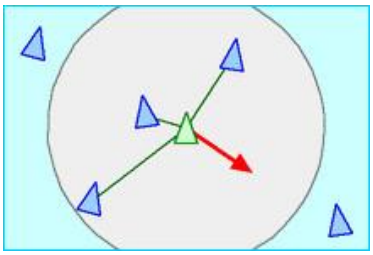

Figure 7: Règle de séparation

6.2.2 Cohésion

Cette règle permet d'éviter que les différents éléments d'un groupe ne s'éloignent trop conduisant à la disparition de ce groupe.

On peut exprimer cette force de la façon suivante :

$$G = \sum_{O_j \in voisinage(O_i)} P_j$$

et

$$\vec{F_c} = seek(P_i, G)$$

Autrement dit la force de cohésion $\vec{F_c}$ est la force de steering qui permet à l'élément O_i de rejoindre le barycentre des éléments appartenant à son voisinage par application de la règle de steering `seek`.

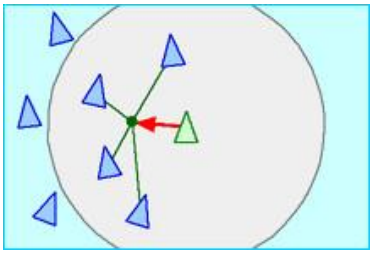

Figure 8: Règle de cohésion

6.2.3 Alignement

Dans un groupe, un moyen de garder la structure de groupe tout en évitant les collisions consiste à imposer le même mouvement de translation à tous les éléments d'un groupe. On cherche donc à imposer la même vitesse $\vec{v}_m$ à tous les éléments d'un même groupe (en l'occurrence ici les individus du voisinage de O_i). .

$$\vec{v}_m = \frac{1}{||voisinage(O_i)||} \sum_{O_j \in voisinage(O_i)} \vec{v}_i$$

La force $\vec{F_a}$ correspond à la force à appliquer pour passer de la vitesse $\vec{v}_i$ de l'objet O_i à la vitesse $\vec{v}_m$.

$$\vec{F_a} = \vec{v}_m - \vec{v}_i$$

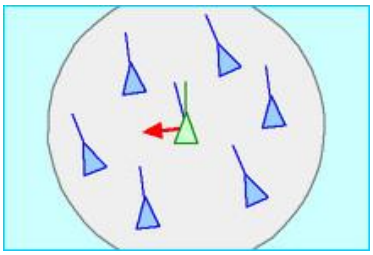

Figure 9:

.

6.2.4 Combinaison des règles

Comme indiqué plus haut les forces résultantes de ces trois règles élémentaires sont combinées linéairement :

$$\vec{F} = k_s\vec{F_s} + k_c\vec{F_c} + k_a\vec{F_a}$$

avec $k_s + k_c + k_a = 1$.

L'affectation des coefficients k_s, k_a, k_c peut être soit global (le même pour toutes les entités du groupe) soit local (choix individuel pour chaque entité du groupe).

Toujours est il que le choix de ces coefficients a une conséquence sur le comportement émergent de groupe : en favorisant un coefficient au détriment des autres on favorise un aspect comportemental plutôt qu'un autre.

Par exemple si k_s est important il y aura moins de collisions entre les entités mais l'aspect de groupe va s'atténuer.

6.3 Exercices

Exercice 1

Créez un groupe de 10 entités qui cont représenter des visiteurs d'un musée. Appliquez leur les règles décrites ci-dessus. Faîtes en sorte que ces visiteurs non seulement aient un comportement de groupe "crédible" mais également qu'ils restent confinés dans une région (le hall de votre musée par exemple). Faîtes en sorte que chaque membre du groupe s'oriente dans le sens de son déplacement.

Exercice 2

Faîtes en sorte que les membres du groupe s'écartent de la caméra virtuelle quand celle-ci est trop proche d'eux.

Exercice 3

On ajoute au groupe de visiteurs une entité distincte : un guide. Celui-ci se déplace dans une région donnée (par exemple le hall de votre projet) selon une trajectoire qui lui est propre (définie a priori ou calculée au fur et à mesure de la simulation). Faîtes en sorte que le groupe de visiteurs suivent (a) le guide (b) un point situé derrière le guide. Quelle différence constatez vous.

Exercice 4

Faites en sorte que le groupe, suivant le guide, s'arrête devant un tableau et se mette à le regarder.

Exercice 5

Faites en sorte que le groupe, suivant le guide, puisse passer du hall à une des salles du musée et ceci de façon aussi crédible que possible.